1. Identificación de Pares: Enumera tres claves (keys) presentes en el nivel superior del objeto JSON.

Tipos de datos:

"R1-Core" string,

True booleano

45 int,

[{ "name": "GigabitEthernet0/0", "description": "Link to WAN", "enabled": true, "mtu": 1500 }, { "name": "GigabitEthernet0/1", "description": null, "enabled": false, "mtu": 1500 }] Lista

2. Tipos de Datos: Identifica un ejemplo de cada uno de los siguientes tipos en el código: String, Number, Boolean y Nul

"R1-Core" string, 45 int, True booleano, null Nul,

3. ¿Qué errores de sintaxis impiden que este JSON sea válido según los estándares de red y programación? Selecciona las dos opciones correctas.

A) El valor de 'vlan_id' debería estar entre comillas dobles obligatoriamente.

B) Las claves como 'vlan_id' no deben llevar guiones bajos.

C) Existe una coma final (trailing comma) después del valor de 'description'.

D) Faltan comas para separar los pares clave-valor después de 'vlan_id' y 'name'.

4. Reflexión Técnica: Explica por qué es crítico que un controlador como Cisco DNA Center reciba un JSON perfectamente validado antes de aplicar cambios en la red

La sintaxis debe ser correcta ya que si contienen errores la ejecución del programa para y no se realiza la programación.

Actividad 2

1. Escribe el pseudocódigo o completa el script de Python necesario para procesar la variable inventario_json (que contiene el texto anterior). El script debe filtrar e imprimir únicamente los hostnames de los dispositivos cuyo estado sea 'down' .

```
for d in inventario:
    if d["status"] == "down":
        print(d["hostname"])
```

Parte 2: Comparativa de Formatos (JSON vs. XML)

En redes modernas, REST APIs suelen preferir JSON por ser más ligero, pero muchos sistemas heredados o protocolos como NETCONF utilizan XML. Observa cómo se estructuran los mismos datos en ambos formatos.

Formato JSON	Formato XML

{
"hostname": "R1",
"status": "up"
}
	xml
	<device>
	<hostname>R1</hostname>
	<status>up</status>
	</device>

2.